

一個在動態環境下的最短路徑搜尋演算法*

陳弘升 劉傳銘[†] 廖明傑
台北科技大學資訊工程系
{t6598039, cmliu[†], s3598005}@ntut.edu.tw

摘要

「最短路徑問題」被探討已行之有年，然而隨著時間的演進，最短路徑問題探討的範疇逐漸從靜態圖形轉移至動態圖形，而至今能夠實際有效地應用在動態圖形上之最短路徑演算法仍屬少數。本文中所探討的正是一種補足 Dijkstra 演算法與 A* 演算法等靜態最短路徑演算法在動態圖上搜尋的時間無法有效被降低的「動態最短路徑演算法」。首先，我們透過文獻探討「最短路徑規劃問題」的發展，以了解 Dijkstra 演算法與 A* 演算法。接著筆者提出本文所要探討的動態最短路徑規劃問題與解決方法，並提供演算法實驗結果加以檢驗演算法的真實效率。透過實驗數據統計圖表加以比較 Dijkstra 演算法、A* 演算法與本文所提出之「動態最短路徑演算法」，實驗結果可說明本文所提出之「動態最短路徑演算法」能有效補足 Dijkstra 演算法與 A* 演算法在動態環境中的效率。

1. 緒論

最短路徑問題是在圖形理論中最佳化問題之一，其應用問題主要為最短路徑規劃以及最短時程規劃。以實例來說：從甲地到乙地，哪一個行程最省時？哪一個行程最省錢？哪一個行程最快？都是為最短路徑典型應用問題。

依照最短路徑問題本質上的不同，最短路徑問題，可以被分類為下列四種[1]：

- I. Single-Pair Shortest-Path, SPSP：已知起點與終點的最短路徑問題。
- II. Single-Source Shortest-Paths, SSSP：指定起點，並要求尋找該起點至其他所有節點最短路徑問題。
- III. All-Pairs Shortest-Paths, APSP：全局的路徑問題：在圖中任意兩點，尋找兩點之間的最短路徑。
- IV. Single-Destination Shortest-Paths, SDSP：指定終點，並要求尋找所有節點至該節點的最短路徑問題。

上述的四類問題，均有已知的最短路徑演算法可以對應與解決。如：雙向搜尋法(Bi-directed)[2]可解決上述中第 I 類問題；Dijkstra 最短路徑演算法等可解決上述中第 II 類問題；Floyd-Warshall[1]可解決上述中第 III 類與第 IV 類問題。

上述的大部分演算法多以靜態資料為主，而隨著無線行動網路的發達與適地性服務的普及，人們對於即時的地理資訊的需求與日俱增，如何提供即時的動態資訊服務將是一重要的研究課題，而動態的資料也為最短路徑問題帶來新的挑戰。現有靜態最短路徑演算法中，均需使用圖形資訊來計算及預測最短路徑，若在動態圖形環境下應用這些演算法，則必須重複的計算與預測路徑，導致時間成本的增加以及運算效能的劇降。針對於此，已有許多動態最佳路徑演算法[3][4][5][6][7][8][9][10][11]被提出，這些論文多以數學模型取代實驗數據證明該演算法的效能，同時，也只有極少數的論文[12][13][14]有提供相關實驗數據來比較各演算法的不同。

在本文中，筆者將提出一個適用於泛型地理資訊應用的「動態最短路徑演算法」來克服上述演算法所遭遇之問題。透過使用引導表的方式，將路徑計算與預測所需之時間複雜度，可以被大幅地降低，相較於其他最短路徑演算法，可大幅減少重複計算與預測路徑所帶來的時間成本。且從實驗結果可以得知，本文所提出的動態最短路徑演算法，在邊權重值影響比例越高之動態圖形上，能取得明顯優於其他演算法的演算效率，而隨著圖形的規模增加與邊影響的比例越高與其他演算法的效率差距越大。

本文接下來的章節順序與內容如下：在第二節，我們探討現有最短路徑規劃演算法的發展並了解其不足之處；第三節，介紹本文所提出的動態路徑演算法所著眼與欲解決之問題；第四節，深入分析與說明本文所提出的動態路徑演算法；第五節，將本文所提出的動態路徑演算法、Dijkstra 演算法與 A* 演算法一同實驗，並深入剖析其實驗結果；第六節總結本文所提出之動態路徑演算法的特性並提出其適合之應用領域。

2. 文獻探討

在現有最短路徑演算法中，分別以 Dijkstra 的演算法與 A* 為靜態最短路徑演算法的代表。在 Dijkstra 演算法上，存在許多以更有效的資料結構的實作方式來增加其查詢效率，如 [15][16][17][18][19] 等。其中 [19] 使用 Fibonacci Heap 實作 Dijkstra 的優先權佇列 Q ，使得 Dijkstra 在節點數為 n 個，邊數為 m 條之圖形中，SSSP 的效率達到 $O(m+n\log n)$ ，同時 APSP 的效率達到 $O(mn+n^2\log n)$ 。在 A* 演算法中，更有許多探討啟發式函數的論文與書籍來討論如何增加 A* 演算法的效率。在後續，更有許多演算法都以上述二個演算法與效率參數作為基礎，企圖尋找出更有效的演

* 感謝國科會對本研究論文的補助，計畫編號：NSC 95-2218-E-027-014。

算法。

除了以解決最短路徑問題為目標的演算法外,有許多方法已經被提出用以改善最短路徑演算法效能。例如,目標導向搜尋法(Goal-directed)[20]、預先計算結果搜尋法(Pre-computing)[21][22][23]、階層式搜尋法(Hierarchy)[24][25][26]等,均是以提升演算法效率而設計的方法。

在近代,計算機、行動裝置與通訊網路改變了人們的生活習慣,進而產生了「無所不在的資訊(Ubiquitous Information)」的觀念,同時亦改變了許多領域的應用環境。以交通系統為例,過去以靜態地圖做為路徑規劃的參考,也必須在路徑規劃時一併考慮即時路況來進行運算。因此,最短路徑演算法在過去以發展於靜態圖型為主,在「資訊到處存在」觀念發展後,開始逐漸轉向動態圖型,在近代,動態最短路徑演算法逐漸成為主要探討目標。

然而,無論 Dijkstra 演算法或 A*演算法,均需使用圖形資訊作為其路徑計算與預測之根據。當將其套用至動態圖形時,由於動態圖形的變化特性,使得上述二演算法在路徑計算與預測上,所花費的時間成本隨著圖形的規模以及圖形動態變化的比例增加而快速上升。Dijkstra 演算法在擴展最短路徑時,將因圖形的動態變化使得其最短路徑的擴展方式產生變化因此影響計算結果的時間。而 A*演算法,則因圖形的動態變化使得其路徑評估的準確度下降。在本文中所提出的動態路徑演算法,即著墨於如何在動態圖形上改善上述演算法的效率。

3. 問題描述

在動態圖形中,最短路徑的規劃需要考慮的二點問題如下:

- 有效且正確的計算最短路徑:
- 由於圖形動態的改變,如何有效且正確的計算最短路徑為演算法的主要目標。
- 圖形改變對演算法的影響:

由於圖形動態的改變,會影響演算法使用於計算與估計最短路徑之輔助判斷資訊的可用度。如何降低圖形改變對演算法的影響,可以有效提升演算法在面對動態圖形的演算效率。

在本文中,目的為提出一動態最短路徑演算法,針對上述二點問題加上克服並改善,最後以實驗證明其實際運用價值。

4. 解決方案

在本文中,筆者將提出一個適用於環境的最短路徑演算法,稱為動態最短路徑演算法(Dynamic Shortest Path algorithm, DSP)。此演算法所考慮的模型為邊權重值會隨時間變化之動態圖形,而基於地理資訊應用的資料特性,如路況發生等,任一時間點 i 之圖形,其任一邊權重值必會大於或等於該邊初始權重值,以得以調整最短路徑。為了有效且即時地計算出最短路徑, DSP 演算法使用「預先計算結果演算法」先行處理初始圖形並產生用以計

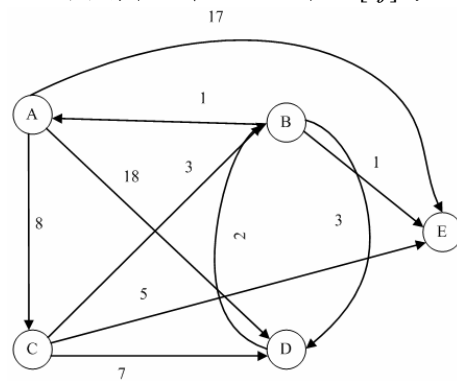
算動態最短路徑之輔助資訊。當 DSP 演算法需要真正計算最短路徑時,使用「目標導向搜尋演算法」,以先前輔助資訊為導向基準並與當下時間點的圖形狀態一同計算出該時間點上的最短路徑。

4.1 DSP 演算法

接下來,本文依 DSP 演算法的計算特性,分別以「預先計算最佳解」與「最短路徑計算過程」二階段,深入解釋 DSP 演算法如何產生動態最短路徑。

A. 預先計算最佳解

首先,對於圖中所有節點,使用 Dijkstra 計算全節點配對最佳的解,同時紀錄節點與其它節點間最短路徑的總權重值於引導表(Guide Table, GT)中。引導表 $GT[i,j]$ 表示節點 i 到節點 j 存在一條最短路徑的路徑長(該路徑上所有邊的權值加總),如果二點之間不存在一最短路徑則 $GT[i,j]$ 為 ∞ 。



圖一、一個加權有向圖。

以圖一的加權有向圖為例,圖中一共有 5 個節點(A,B,C,D,E),每一個節點都以 Dijkstra 演算法計算以該節點為根節點(Root)的最小生長樹(Minimal Spanning Tree, MST)。計算最小生長樹時,使用一暫存的陣列,儲存起點至其他節點的最小距離,以 $d[node]$ 來表示,初始值均為 ∞ ,例如 $d[A]$ 為起點到 A 節點的最小距離。以節點 A 為例:首先將 A 到 A 的距離: $d[A]$ 與引導表 $GT[A,A]$ 設定為 0,同時計算所有與 A 相鄰的節點($Adj[A]$): C、D、E 與節點 A 的距離,分別為 $d[C]=8$, $d[D]=18$, 與 $d[A]=17$ 。接下來比較各節點的 d 值,以鄰近節點中與 A 距離最近的節點 C 為中繼點,將 $d[C]$ 加入到 GT,也就是 $GT[A,C]=8$,同時計算所有 C 的鄰近節點,經由節點 C 到達 A 之距離,如果經由節點 C 可以讓某節點 V 之 d 值變小,例如 $d[C]+l(C,E)_0 = 13 < d[E]$, ($l(C,E)_0$ 為沒有路況下的連接節點 C 與 E 之邊),則將 $d[E]$ 修改成 13,再將所有節點中 d 值最小者加入到引導表;持續這些步驟,直到所有節點均加入至引導表為止才結束節點 A 的計算。然後換至另一點,如節點 B 來計算 MST,直到道路上所有節點均完成 MST 的計算為止。表一則是由圖一的加權有向圖所產生的權值引導表,在此引導表 $GT[i,j]$ 中,表上方所列節點為起點,表左方所列節點為終點。

B. 最短路徑計算(搜尋)過程

對於每一次查詢最短路徑中起點 s 與終點 t ,

本演算法使用目標導向演算法計算最短路徑解，同時目標導向演算法中對於選取節點所採用的評估

終點 起點	A	B	C	D	E
A	0	1	4	3	∞
B	11	0	3	2	∞
C	8	9	0	11	∞
D	14	3	6	0	∞
E	12	1	4	3	0

表一 由圖一產生之引導表。

方式則使用引導表來作計算，如由節點 x 到 t 的路徑的預測長度為 $GT[x, t]$ 。在路徑計算的過程中，使用一暫存的優先佇列 Q ，存放待觀察之節點 x ，計算 s 到 x 之間的最短路徑長度與預測 x 到 t 的權值總合，最小者作為下一個已觀察過節點 v 來執行查詢過程。當終點被列入觀察之節點時，演算法結束查詢同時回傳查詢的結果。

在這裡，與 A* 演算法有類似的模式，使用一個評估的啟發式函數(Heuristic function)，函數的輸出即為查詢引導表所對應出的靜態圖上之最短路徑總長度。

我們以查詢路徑 A~E 為例子作說明：

- ◆ Step1：以 A 為中心，計算鄰近節點 C、D、E

$$f[C] = g(A) + l(A, C)_i + GT[C, E] = 0 + 8 + 4 = 12, \\ \pi(C) = A;$$

$$f[D] = g(A) + l(A, D)_i + GT[D, E] = 0 + 18 + 3 = 21, \\ \pi(D) = A;$$

$$f[E] = g[A] + l(A, E)_i + GT[E, E] = 0 + 17 + 0 = 17, \\ \pi(E) = A;$$

其中 $f[V]$ 為 A~E 路徑中，透過節點 V 的預測總路徑長， $g(V)$ 為節點 V 到節點 A 的實際路徑長， $l(x, y)_i$ 為時間 i 時連結節點 x 與節點 y 之間邊的權值， $\pi(V)$ 則表示 A~E 路徑中連結節點 V 的上一節點資訊

- ◆ Step2：因為 $f[C]$ 最小，故以 C 為主，計算鄰近節點 B、D、E

$$f[B] = g(C) + l(C, B)_i + GT[B, E] = 8 + 3 + 1 = 12, \\ \pi(B) = C;$$

$$f[D] = g(C) + l(C, D)_i + GT[D, E] = 8 + 7 + 3 = 18, \\ \pi(D) = C;$$

$$f[E] = g(C) + l(C, E)_i + GT[E, E] = 8 + 5 + 0 = 13, \\ \pi(E) = C;$$

- ◆ Step3：因為 $f[B]$ 最小，故以 B 為主，計算鄰近節點 D、E

$$f[D] = g(B) + l(B, D)_i + GT[D, E] = 11 + 2 + 3 = 16, \\ \pi(D) = B;$$

$$f[E] = g(B) + l(B, E)_i + GT[E, E] = 11 + 1 + 0 = 12, \\ \pi(E) = B;$$

- ◆ Step4：因為 $f[E]$ 最小，所以 E 成為下一個中心節點，但是 E 也是終點，故演算法停止，

傳回路徑為 A~C~B~E。

演算法虛擬程式碼，如圖二所示，其中 G 代表圖本身，L 代表 G 的所有節點數，s 為起點，t 為終點，GT 為引導表：

$SRC(G, L, s, t, GT)$	cost	times
1 for each node $v \in V[G]$	c1	L
2 do $f[v] \leftarrow \infty$	c2	L
3 $\pi[v] \leftarrow NIL$	c3	L
4 $g[s] \leftarrow 0$	c4	1
5 $S \leftarrow \emptyset$	c5	1
6 $Q \leftarrow V[G]$	c6	1
7 $check \leftarrow false$	c7	1
8 while $check = false$	c8	L
9 do $u \leftarrow EXTRACT-MIN(Q)$	c9	L
10 if $u = t$	c10	L
11 then $check \leftarrow true$	c11	1
12 else $S \leftarrow S \cup \{u\}$	c12	L-1
13 for each node $v \in Adj[u]$	c13	L(L-1)
14 if $f[v] > g[u] + l(u, v)_i + GT[v, t]$	c14	L(L-1)
15 then $f[v] \leftarrow g[u] + l(u, v)_i + GT[v, t]$	c15	L(L-1)
16 $g[v] = g[v] + l(u, v)_i$	c16	L(L-1)
17 $\pi[v] \leftarrow u$	c17	L(L-1)

圖二、DSP 演算法中，最短路徑計算(搜尋)虛擬碼與各敘述執行所需的時間，其中 c_i 為一常數，代表各敘述所需的執行時間。

由於 DSP 演算法利用了目標導向搜尋演算法的特性，在計算最短路徑的過程中，使用引導表來預測路徑總長，藉此加快路徑查詢的速度。

4.2 演算法正確性

引用[27]定理 1： $\hat{h}(n) \leq h(n)$ ， $h(n)$ 代表預測路徑之真實長度， $\hat{h}(n)$ 預測路徑之預測長度。則其演算法必然可以計算出最短路徑。

考慮動態圖形中，任意時間點 i 之圖形，其預測路徑之真實長度以 $h(n)$ 表示，預測路徑之預測長度以 $\hat{h}(n)$ 表示。由於 DSP 演算法計算最短路徑的過程中，使用引導表來預測路徑總長，並且：

1. 引導表以初始圖形使用 Dijkstra 演算法計算產生。
2. 任一時間點 i 之圖形 G_i ，其任一邊權重值必會大於或等於該邊於時間點 0，即 G_0 之初始權重值。

故可知對任何時間點 i ， $\hat{h}(n) \leq h(n)$ 均會成立，因此 DSP 演算法必然可以計算出最短路徑。

4.3 演算法時間複雜度

DSP 演算法中，最短路徑計算(搜尋)的時間複雜度， $T(L)$ ，的計算可先將虛擬碼中各敘述執行所需的時間一一展開，如圖二所示。DSP 演算法在最差的狀況，每個點均與所有其他節點相連，此時間複雜度為 $O(L^2)$ 。但在一般情況下，圖上的任一點僅會與周圍的數個點相連，假設圖中節點的平均扇出數(out-degree)為 c_{Ave} ，故可以將演算法執行所花費的時間改寫如下：

$$T(L) = c_1 L + c_2 L + c_3 L + c_4 + c_5 + c_6 + c_7 + c_8 (L+1) \\ + c_9 L + c_{10} L + c_{11} + c_{12} (L-1) + c_{13} c_{Ave} (L-1) \\ + c_{14} c_{Ave} (L-1) + c_{15} c_{Ave} (L-1) + c_{16} c_{Ave} (L-1)$$

$$+ c_{17}c_{Ave}(L-1) = O(L)。$$

故可以得知，在一般狀況下，DSP 演算法的時間複雜度為 $O(L)$ 。

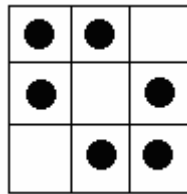
5 實驗與數據

在本段落中將介紹 DSP 演算法的實驗內容。在實驗中，使用一隨機產生的圖，並在這個圖上執行 Dijkstra 演算法、A*演算法(使用以歐式空間距離為基礎的 Heuristic Function)和 DSP 演算法，最後蒐集實驗結果並比較之。

5.1 實驗環境

Step 1：產生隨機圖

產生隨機圖的方法參考[28]，首先決定圖的節點數量為 m ，並畫出一表格，方型表格其欄與列數量均為 n ，使得 $n^2 \geq m$ 。接下來，在表格內隨機選取 m 格填入節點，其節點坐標以 (r, c) ， r 為列編號， c 為欄編號，如圖三所示。



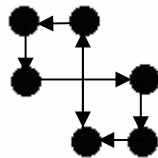
圖三 隨機節點圖範例($n=3, m=6$)

Step 2：產生有向邊

尋訪表格中的每一節點並隨機建立指向鄰近節點連線的有向邊，每一個節點至少必須存在一有向邊指向鄰近節點。

Step 3：確定圖中所有節點均能有效被尋訪

檢查是否每一節點是否有連入的邊。若存在一節點無任何連入的邊，則該節點則無法被尋訪，則重新執行 Step 2 直至任一節點至少存在一連入的邊，如圖四所示。在實驗中，使用圖的大小為從 3000 點到 10000 點：



圖四 隨機圖範例

Step 4：產生初始圖形並計算引導表

在隨機圖產生後，設定每一邊的權重值均為 1，並以該圖當作初始圖形使用 Dijkstra 演算法來計算 all pair shortest paths，並紀錄於引導表中。

Step 5：調整邊權重值，模擬不同程度的動態變更
隨機選取邊增加其權重值 10 倍，以測試 DSP 演算法在動態圖上的效率，影響的邊比例((調整邊數量 / 所有邊數量) * 100%)從完全沒變更(0%)到 50%。

5.2 實驗流程

在程式執行時，會先依照 5.1 所述之方法建

立圖形並逐步調整影響的邊比例，並在圖上運行 Dijkstra 演算法、A*演算法和 DSP 演算法。為了提高實驗的精確度，每一實驗所得之數據，均為同一演算法在相同的情況下運行 3000 次後，再取其平均值當作其數據。

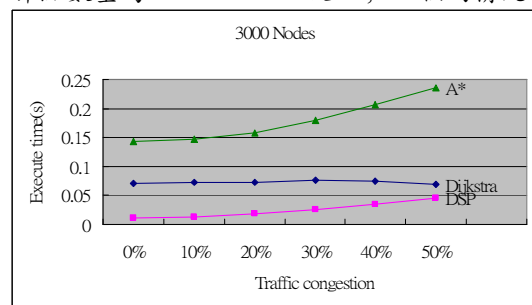
5.3 實驗結果與分析

在實驗中，共設計有三個比較的项目：

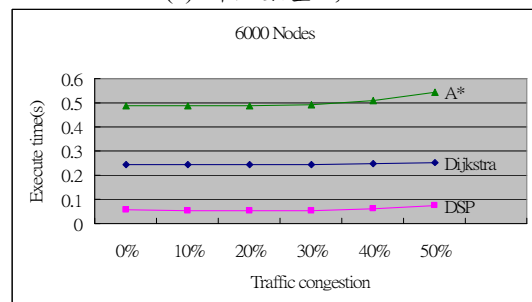
- 不同的邊權重值變化比例，各演算法搜尋最短路徑時間的變化。
- 節點數量增加，各演算法搜尋最短路徑時間的成長曲線變化。
- 不同的邊權重值變化比例，各演算法尋訪過的節點數量變化。

項目 A

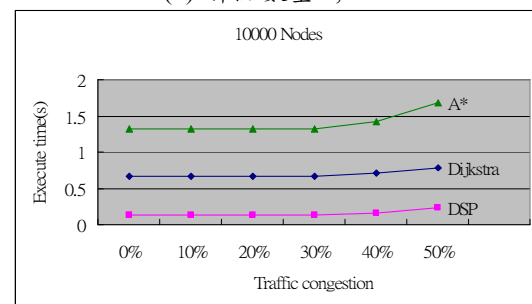
實驗項目 A 的結果如圖五所示，分別展示了在節點數量為 3000，6000 及 10,000 個的情況。



(a) 節點數量 3,000



(b) 節點數量 6,000



(c) 節點數量 10,000

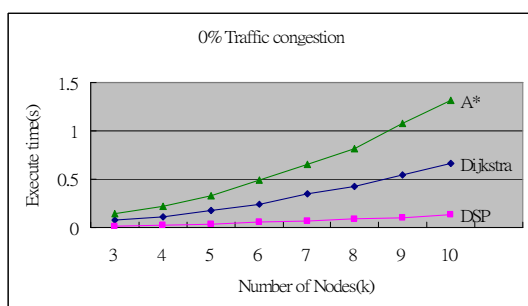
圖五¹、各演算法在不同的邊權重值變化比例下，搜尋最短路徑的時間，節點數量分別為(a) 3000, (b) 6000，及(c) 10,000 個。

A*演算法花費時間最多，其原因為在 A*演算法中，計算估計值的啟發式函數使用歐式空間中

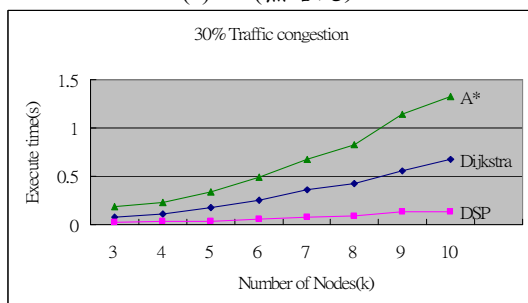
¹ A* 啟發式函數： $h(x,y) = \sqrt{x^2 + y^2}$, (x,y) 表節點坐標。

計算兩點直線距離的方式¹，故所需要花費的計算時間複雜度會增加。如實驗統計圖所示，A*演算法在邊影響比例增加時，所需要較多的運算時間。這是因為 A*演算法中啟發式函數的準確度，會受到邊影響比例所干擾的緣故。

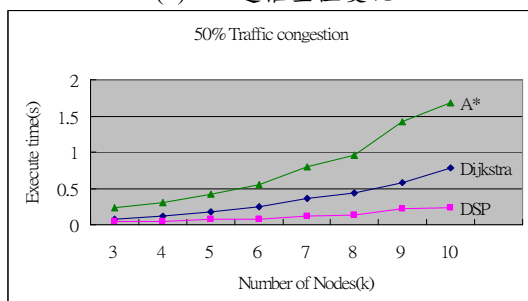
而我們也可以從結果中發現，DSP 演算法的平均搜尋時間總是比 Dijkstra 演算法還來的少。當圖的節點數越多時，DSP 演算法與 Dijkstra 演算法之間的差距會越大，因為 DSP 演算法依據查表的結果可以縮減向外擴散範圍朝終點的方向前進，而查表的時間所花費的時間成本非常低，通常為常數時間。由於 DSP 演算法，使用引導表決定最短路徑擴展方向，降低邊影響比例的干擾，因此在邊影響比例上升時，所受的干擾幅度遠低於 A*演算法，並僅略高於 Dijkstra 演算法。透過觀察實驗結果可以發現，DSP 演算法透過使用引導表大幅提升對於邊權重值改變的抵抗能力，並且取得良好的演算效率。



(a) 0%(無路況)



(b) 30%邊權重值變化



(c) 50%邊權重值變化

圖六、各演算法在節點數量增加時，搜尋最短路徑時間的成長變化，邊權重值變化比例分別為(a) 0%(無路況)、(b) 30%及(c) 50%。

項目 B

在實驗項目 B 中，我們考慮了不同邊權重值變化的比例(即路況發生比例)，圖六呈現的結果其比例分別為 0%(無路況)、30%及 50%路況。目的為比較在不同節點數量時，各演算法的時間複雜度曲線的上升速度。在 A*演算法中，由於在節點數量增加時計算啟發式函數值的次數變多，因此其時間複雜度曲線快速上升。而 Dijkstra 演算法由於並無使用啟發式函數的緣故，因此其時間複雜度曲線上升速度較為平緩。

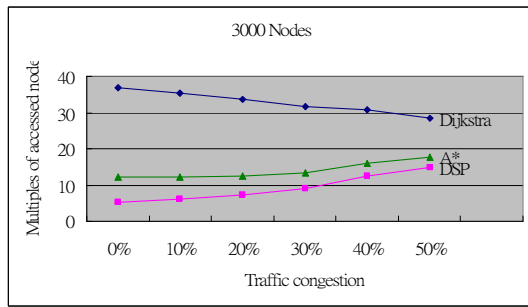
最後，在 DSP 演算法的實驗數據中，可以藉由統計圖表發現，其所需的時間複雜度上升速度遠低於 A*演算法與 Dijkstra 演算法。這是由於 DSP 演算法決定最短路徑擴展方向時，使用查表的方式將所需之時間複雜度減低至常數時間，故其時間複雜度曲線不如 A*演算法般的快速上升，同時因為 DSP 演算法使用引導表有效降低節點數量增加時對於演算法的影響，因此其時間複雜度曲線的上升速度均較 A*演算法與 Dijkstra 演算法緩慢，進而取得在大量節點運算時的優勢。

項目 C

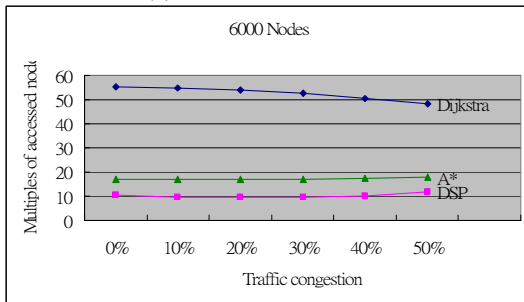
實驗項目 C 在探討各演算法在不同的邊權重值變化時所尋訪過的節點數量變化，其結果如圖七所示。在實驗項目 C 中，透過比較各種演算法在計算最短路徑中所尋訪過的節點數量，來呈現各演算法對於計算最短路徑過程中所做決策的有效程度。對照實驗項目 A 所比較的時間複雜度，我們可以明顯的觀察到 A*演算法雖然在過程中的尋訪節點數量遠低於 Dijkstra 演算法，但受到計算啟發式函數值的影響，並未能降低其時間複雜度，因此 Dijkstra 演算法反而具有良好的效率。

實驗結果中發現，Dijkstra 演算法所需的尋訪節點數並不隨邊影響比例的上升而增加。在經過分析 Dijkstra 演算法的 Greedy 特性後，可以歸因於當邊影響比例上升時，使 Dijkstra 演算法在計算最短路徑過程中早期被考慮的節點數目減少，而原先必須同時考慮的節點則因其邊權重值的增加而被延遲考慮。除此之外，又因為終點節點落於權重值有變化之邊上的機率與邊影響比例相同，故隨著邊影響比例的上升，Dijkstra 演算法在計算最短路徑過程中所需尋訪的節點平均相對較少。由於 Dijkstra 演算法所需尋訪之節點數受其終點落於權重值有變化之邊上的機率與其過程中早期參考節點數量的影響，因此可以得知 Dijkstra 演算法在邊影響比例達 50%時所需尋訪的節點數量為最少。

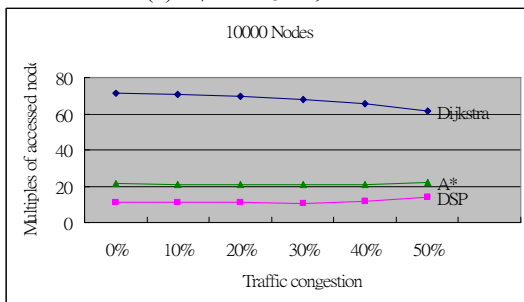
在 DSP 演算法中，由於採用引導表做為最短路徑擴展方向的評估，同時該引導表對於邊權重改善的抵抗力優於 A*演算法中的啟發式函數，故在邊影響比例上升的情況下，更能準確評估最短路徑擴展方向，因此演算過程中所需之尋訪節點數亦少於 A*演算法。



(a) 節點數量 3,000



(a) 節點數量 6,000



(c) 節點數量 10,000

圖七、各演算法在不同的邊權重值變化時所尋訪過的節點數量變化，節點數量分別為節點數量分別為(a) 3000, (b) 6000, 及(c) 10,000 個。

6 結論

DSP 演算法補足 Dijkstra 演算法與 A* 演算法的不足。DSP 演算法利用引導表降低 A* 演算法中啟發式函數帶來的時間成本至線性時間，有時更可達常數時間，並有效降低邊權值變化時對演算法的影響。同時 DSP 演算法亦使用引導表改善 Dijkstra 演算法的無方向特性，使其能夠正確的評估最短路徑擴展方向進而提升演算法效率。未來更計劃以實際地理道路資料來進行實驗探討。

DSP 演算法在邊影響比例上升時，能夠有效降低邊權值變化時對演算法的影響，故在邊權值發生變化的動態圖形上演算最短路徑能夠取得良好的效率。由於其優異的演算效率，使其實作於嵌入式裝置與行動裝置的可行性增加。並因 DSP 演算法所使用的圖形，具有結構上不可變化的特性，因此極適合應用於地理或交通資訊系統等望模環境相對穩定的應用領域。

7 參考文獻

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, "Introduction to Algorithms 2nd Edition", 2001.
- [2] I. Phol, "Bi-directional search." Machine Intelligence, Vol 6, pp. 127-170, 1971.
- [3] V. King, "Fully dynamic algorithms for maintaining all-pairs shortest paths and transitive closure in digraphs." in Proc. Of the 40th Annual Symposium on Foundations of Computer Science, pp. 81-99, 1999.
- [4] S. Pettie, "A new approach to all-pairs shortest paths on real-weighted graphs." Theoretical Computer Science, 312(1), pp. 47-74, 2004.
- [5] S. Pallottino and M. Grazia Scutellà, "A new algorithm for reoptimizing shortest paths when the arc costs change." Operations Research Letters, 31, pp. 149-160, 2003.
- [6] B. Xiao, J. Cao, Q. Zhuge, Z. Shao, and E.H.-M. Sha, "Dynamic shortest path tree update for multiple link state decrements." in Proceedings Of the IEEE Global Telecommunications Conference, 2, pp. 1163-1167, 2004.
- [7] C. Demetrescu and G.F. Italiano, "A new approach to dynamic all pairs shortest paths." Journal of ACM, 51(6), pp. 968-992, 2004.
- [8] M. Thorup. "Worst-case update times for fully-dynamic all-pairs shortest paths." In Proc. Of the 37th Annual ACM Symposium on Theory of Computing (STOC), pp. 112-119, 2005.
- [9] S. Misra and B. John Oommen, "Dynamic algorithms for the shortest path routing problem: Learning automata-based solutions." IEEE Transactions on Systems, Man, and Cybernetics, Part B 35(6): pp. 1179-1192, 2005.
- [10] C. Demetrescu and G.F. Italiano, "Dynamic shortest paths and transitive closure: Algorithmic techniques and data structures." Journal of Discrete Algorithms (JDA), 2006.
- [11] S. Misra and B. John Oommen, "An efficient dynamic algorithm for maintaining all-pairs shortest paths in stochastic networks." IEEE Transactions on Computers, 55(6), pp. 686-702, 2006.
- [12] G. Ramalingam and T. Reps, "An incremental algorithm for a generalization of the shortest-path problem." Journal of Algorithms, 21(2), pp. 267-305, 1996.
- [13] D. Frigioni, M. Ioffreda, U. Nanni, and G. Pasqualone, "Experimental analysis of dynamic algorithms for the single source shortest path problem." in Proc. Of the 1st

- Workshop on Algorithm Engineering (WAE'97), pp. 54-63, 1997.
- [14] C. Demetrescu and G.F. Italiano, "Experimental analysis of dynamic all pairs shortest path algorithms." in Proc. 15th Annual ACM-SIAM Symposium on Discrete Algorithms, pp. 362-371, 2004.
 - [15] B. V. Cherkassky, A. V. Goldberg, and T. Radzik, "Shortest paths algorithms: Theory and experimental evaluation." in Proc. 5th annual ACM-SIAM symposium on Discrete algorithms, pp. 516 - 525, 1994.
 - [16] F. Benjamin Zhan, "Three fastest shortest path algorithms on real road networks: Data structures and procedures." Journal of Geographic Information and Decision Analysis, 1(1), pp. 69-82, 1997.
 - [17] R. Jacob, M.V. Marathe, and K. Nagel, "A computational study of routing algorithms for realistic transportation networks." Journal of Experimental Algorithmics (JEA), 4(6), 1999.
 - [18] F. Schulz, D. Wagner, and K. Weihe, "Dijkstra's algorithm on-line: an empirical case study from public railroad transport." Journal of Experimental Algorithmics (JEA), 5(12), 2000.
 - [19] M.L. Fredman and R.E. Tarjan, "Fibonacci heaps and their uses in improved network optimization algorithms." Journal of the ACM, 34(3), pp. 596-615, 1987.
 - [20] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths." IEEE Transactions on System Science and Cybernetics, 4, pp. 100-107, 1968.
 - [21] D. Wagner and T. Willhalm, "Geometric speed-up techniques for finding shortest paths in large sparse graphs." in Proc. Of the 11th European Symposium on Algorithms (ESA 2003) , 2832 of LNCS, pp. 776-787, 2003.
 - [22] R. Gutman, "Reach-based Routing: A new approach to shortest path algorithms optimized for road networks." in Proc. Of the 6th International Workshop on Algorithm Engineering and Experiments, pp. 100-111, 2004.
 - [23] R. H. Möhring, H. Schilling, B.Schütz, D. Wagner, and T. Willhalm, "Partitioning graphs to speed up Dijkstra's algorithm." in Proc.of the Experimental and Efficient Algorithms, 4th International Workshop, WEA 2005, pp. 189- 202, 2005.
 - [24] N. Jing, Y.W. Huang, and E. A. Rundensteiner, "Hierarchical path views: A model based on fragmentation and transportation road types." in Proc. Of the 3rd ACM Workshop on Geographic Information Systems, pp. 93-100, 1995.
 - [25] N. Jing, Y.W. Huang, and E. A. Rundensteiner, "Hierarchical encoded path views for path query processing: An optimal model and its performance evaluation." IEEE Transactions Knowledge and Data Engineering, 10(3), 1998.
 - [26] S. Jung and S. Pramanik, "An efficient path computation model for hierarchically structured topographical road maps." IEEE Transactions Knowledge and Data Engineering, 14(5), 2002.
 - [27] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths." IEEE Transactions on System Science and Cybernetics, 4, pp. 100-107, 1968.
 - [28] Y.-W. Huang, N. Jing, and E. A. Rundensteiner, "Path Queries for Transportation Networks: Dynamic Reordering and Sliding Window Paging Techniques", Journal of ACM, 1997.